



TECNOLÓGICO
NACIONAL DE MÉXICO



INSTITUTO TECNOLÓGICO DE NUEVO LAREDO

INGENIERIA EN SISTEMAS COMPUTACIONALES

Aplicaciones Descentralizadas

UNIDAD 2



Práctica 1

Geth Ethereum



Docente: Ing. Luis Daniel Castillo García

Integrantes del equipo 10

Gerardo Alberto Cruz Rojo

#19100163

Juan Hernández Rosas

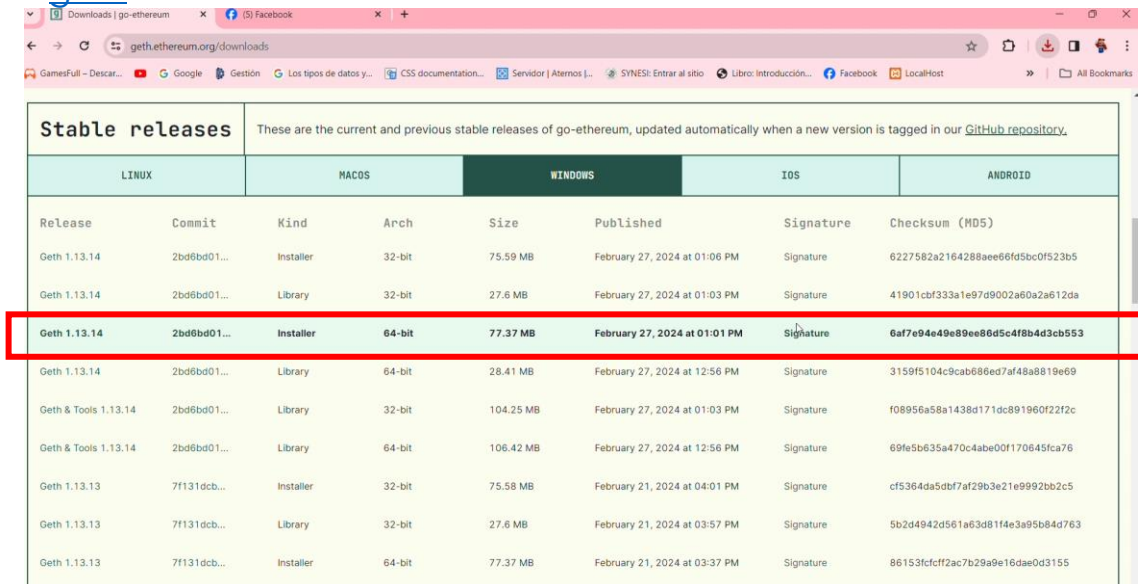
#19100199

Nuevo Laredo, Tamps

18/03/2024

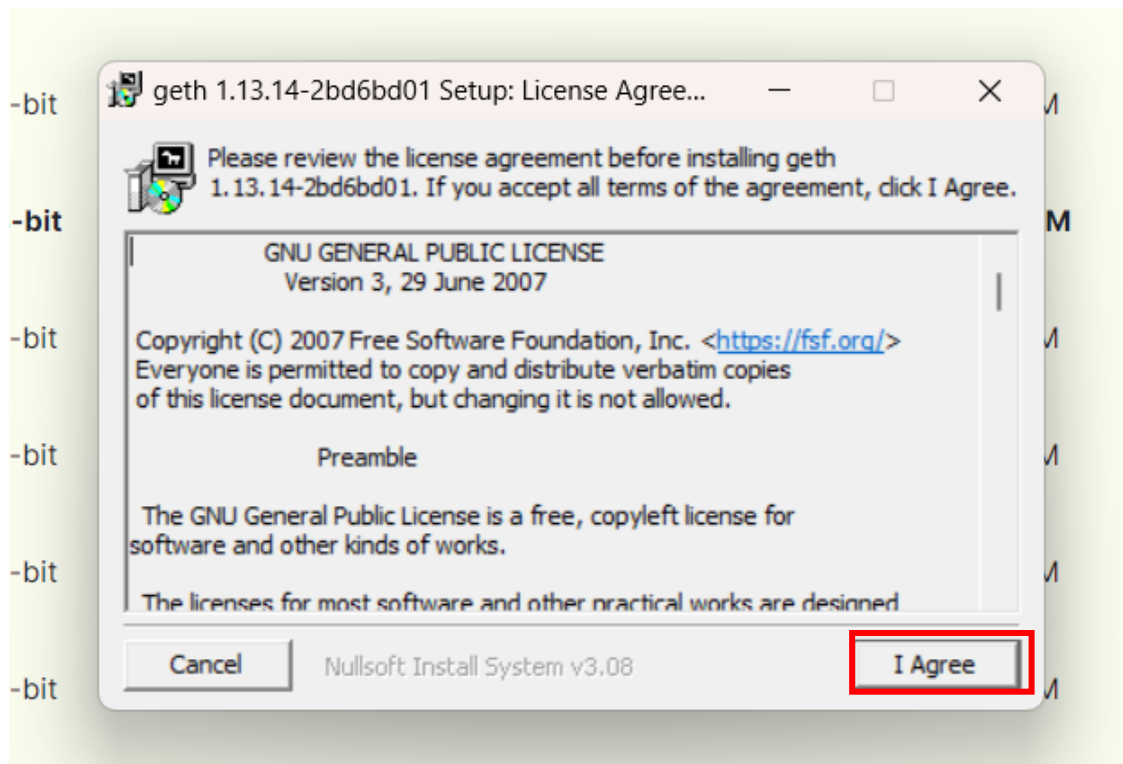
1. Descargar e instalar geth en

<https://geth.ethereum.org/docs/getting-started/installing-geth>

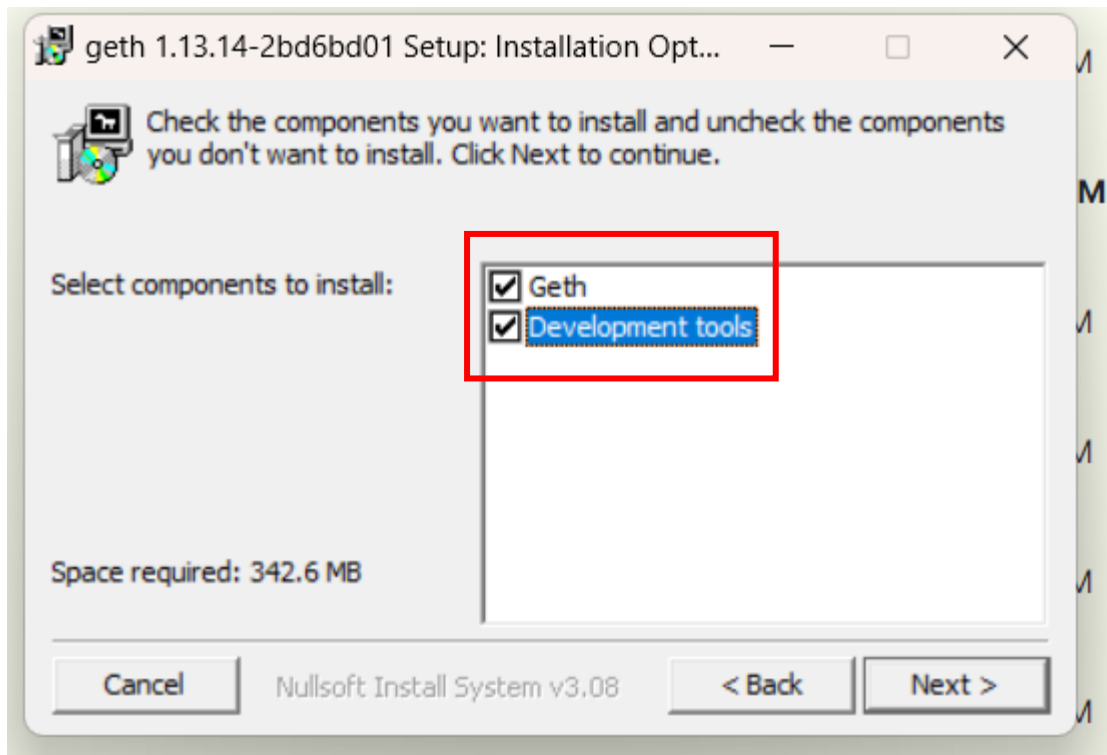


Stable releases		These are the current and previous stable releases of go-ethereum, updated automatically when a new version is tagged in our GitHub repository .					
		LINUX	MACOS	WINDOWS	IOS	ANDROID	
Release	Commit	Kind	Arch	Size	Published	Signature	Checksum (MD5)
Geth 1.13.14	2bd6bd01...	Installer	32-bit	75.59 MB	February 27, 2024 at 01:06 PM	Signature	6227582a2164288ae66fd5bc0f523b5
Geth 1.13.14	2bd6bd01...	Library	32-bit	27.6 MB	February 27, 2024 at 01:03 PM	Signature	41901cbf333a1e97d9002a60a2a612da
Geth 1.13.14	2bd6bd01...	Installer	64-bit	77.37 MB	February 27, 2024 at 01:01 PM	Signature	6af7e94e49e89ee86d5c4f8b4d3cb553
Geth 1.13.14	2bd6bd01...	Library	64-bit	28.41 MB	February 27, 2024 at 12:56 PM	Signature	3159f5104c9cab686ed7af48a8819e69
Geth & Tools 1.13.14	2bd6bd01...	Library	32-bit	104.25 MB	February 27, 2024 at 01:03 PM	Signature	f08956a58a1438d171dc891960f22f2c
Geth & Tools 1.13.14	2bd6bd01...	Library	64-bit	106.42 MB	February 27, 2024 at 12:56 PM	Signature	69fe5b635a470c4abe00f170645fca76
Geth 1.13.13	7f131dcb...	Installer	32-bit	75.58 MB	February 21, 2024 at 04:01 PM	Signature	cf5364da5dbf7af29b3e21e9992bb2c5
Geth 1.13.13	7f131dcb...	Library	32-bit	27.6 MB	February 21, 2024 at 03:57 PM	Signature	5b2d4942d561a63d81f4e3a95b84d763
Geth 1.13.13	7f131dcb...	Installer	64-bit	77.37 MB	February 21, 2024 at 03:37 PM	Signature	86153fcfcf2ac7b29a9e16dae0d3155

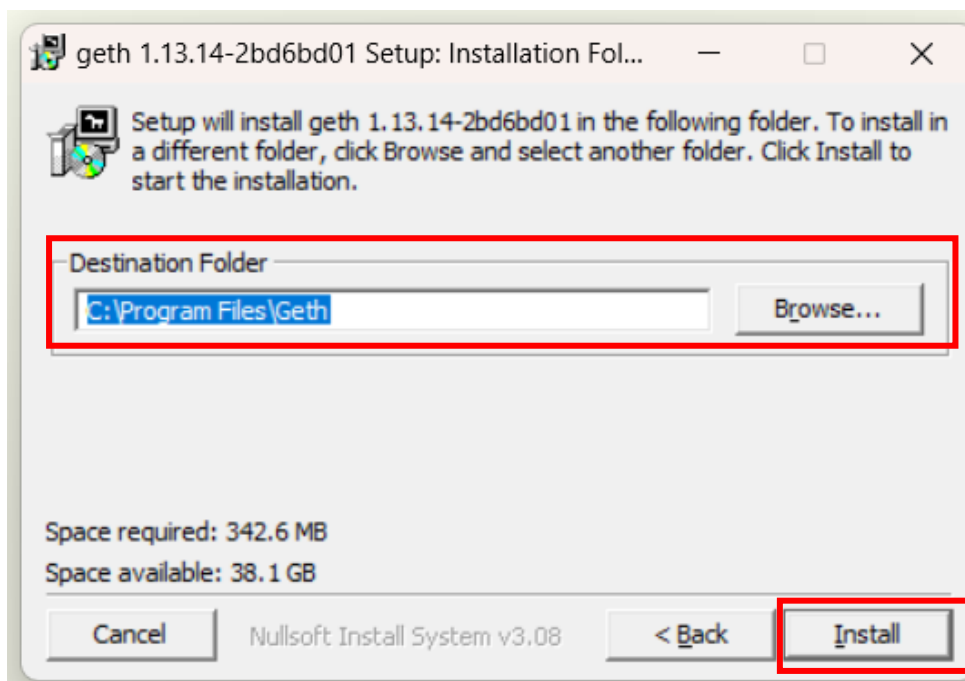
Ingresa a la página y descargue el ejecutable Geth 1.13.14 de 64-bit.



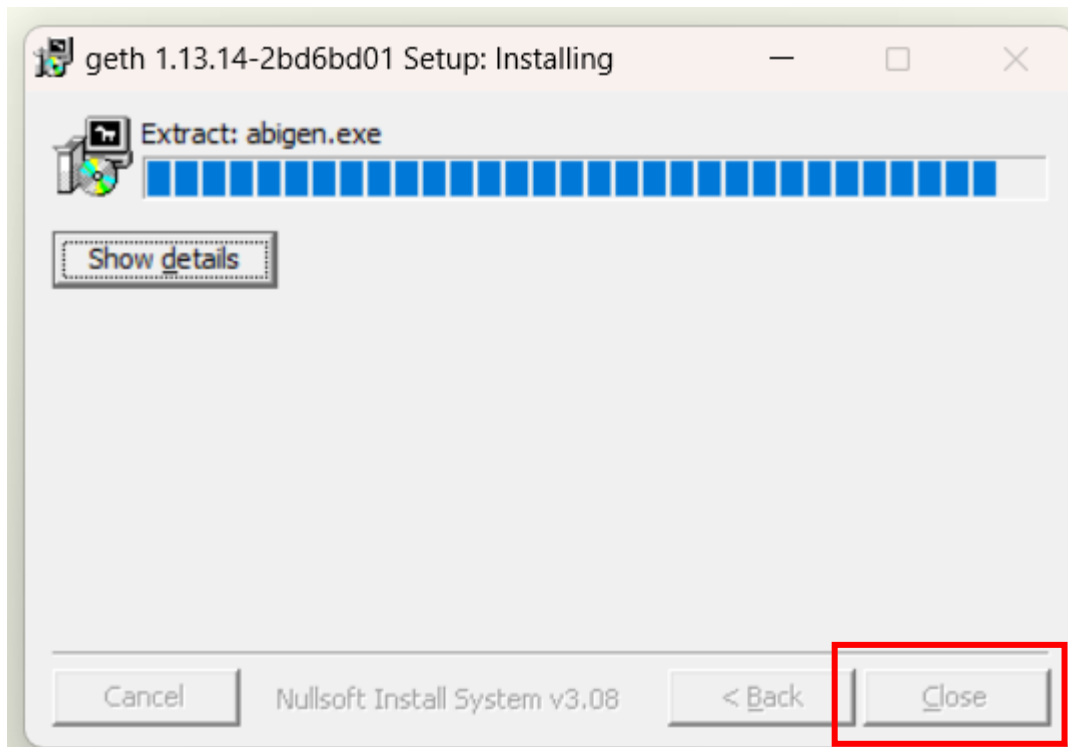
Acepte términos de la licencia.



Di check a Development tools para tener más herramientas que utilizar.

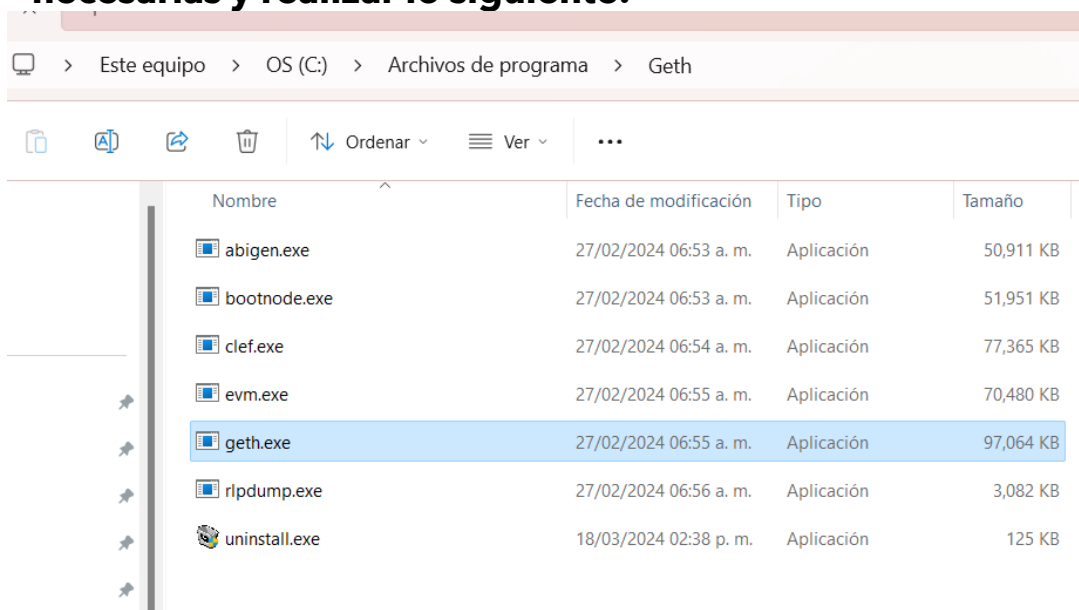


Selecione la ruta donde se iba a instalar.

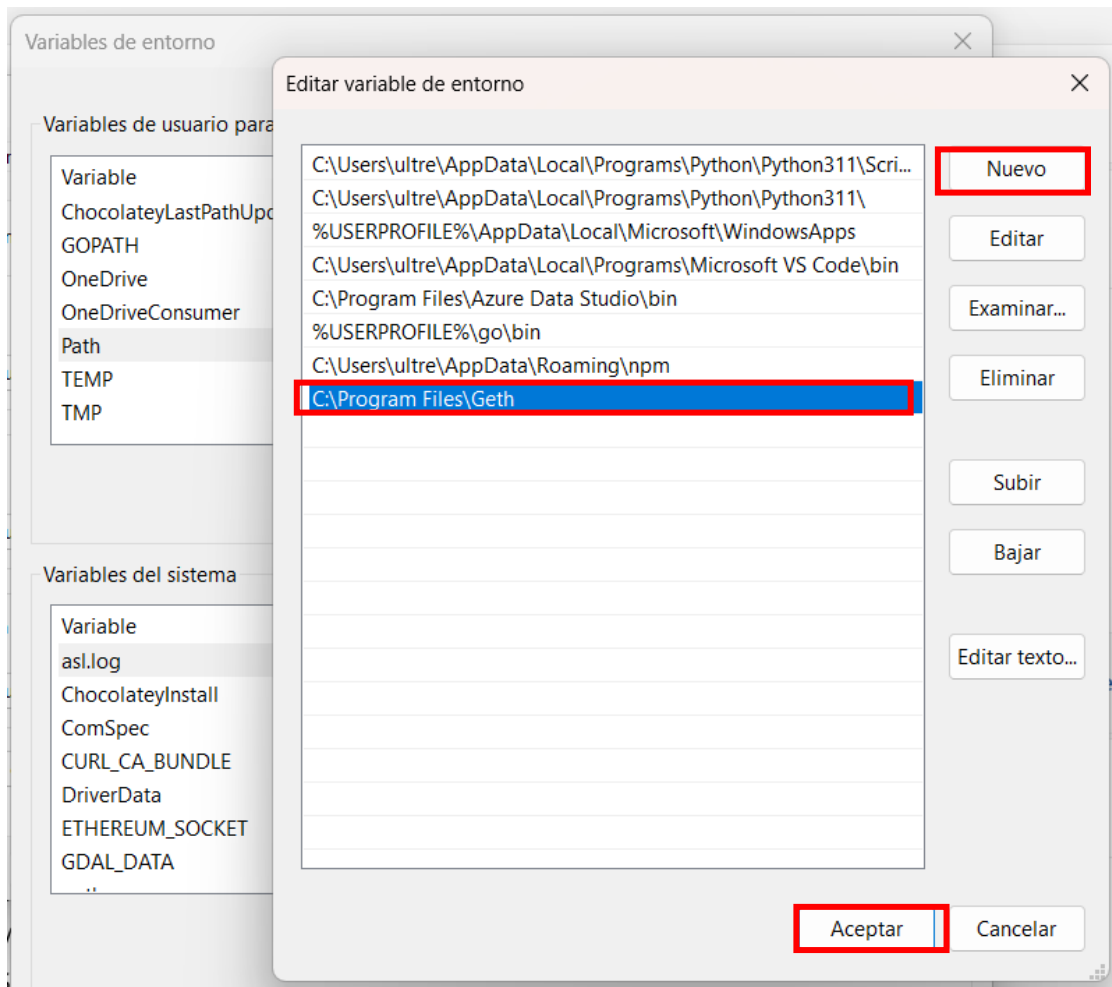


Listo ya que se instaló di a Close para cerrar ventana.

2. Iniciar el nodo en el Net Sepolia utilizando las API necesarias y realizar lo siguiente:



Para iniciar **Geth** me fui a la ruta donde estaba y la copié.



La agregue como variable de entorno en el **Path** para ejecutar donde yo quiera en la terminal.

```
Administrador: Windows Pow x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos
Instale la versión más reciente de PowerSh
PS C:\Users\ultre> Geth|
```

Solo basta con poner **Geth** en la consola de la Terminal.

```
Administrador: Windows Pow  x + v
INFO [03-18|14:53:40.485] Loaded most recent local block      number=0 hash=d4e567..cb8fa3 td=17,179,869,184 age=55
y19h53m
INFO [03-18|14:53:40.485] Initialized transaction indexer      range="last 2350000 blocks"
INFO [03-18|14:53:40.485] Loaded local transaction journal     transactions=0 dropped=0
INFO [03-18|14:53:40.504] Enabled snap sync                   head=0 hash=d4e567..cb8fa3
INFO [03-18|14:53:40.504] Chain post-merge, sync via beacon client
INFO [03-18|14:53:40.504] Gasprice oracle is ignoring threshold set threshold=2
WARN [03-18|14:53:40.508] Unclean shutdown detected           booted=2024-03-13T07:56:35-0500 age=5d6h57m
WARN [03-18|14:53:40.509] Engine API enabled                 protocol=eth
INFO [03-18|14:53:40.509] Starting peer-to-peer node         instance=Geth/v1.13.14-stable-2bd6bd01/windows-amd64/
go1.21.6
INFO [03-18|14:53:40.533] New local node record              seq=1,710,159,826,982 id=54dfb3d257b37029 ip=127.0.0.
1 udp=30303 tcp=30303
INFO [03-18|14:53:40.538] Started P2P networking             self=enode://088e3c7bbdaaeceb560bb0a09d10eb950278ce86
f815a6f16d9f16ddb279b2e0ff4f48dd6b2e054740d8585f8764090db8fdb9636a8637f8bb99524cdb8f56a3@127.0.0.1:30303
INFO [03-18|14:53:40.538] IPC endpoint opened                url=\\.pipe\\geth.ipc
INFO [03-18|14:53:40.538] Loaded JWT secret file             path=C:\\Users\\ultre\\AppData\\Local\\Ethereum\\geth\\jwtse
cret crc32=0xb235a7c2
INFO [03-18|14:53:40.543] WebSocket enabled                  url=ws://127.0.0.1:8551
INFO [03-18|14:53:40.543] HTTP server started                endpoint=127.0.0.1:8551 auth=true prefix= cors=localh
ost vhosts=localhost
INFO [03-18|14:53:42.689] New local node record              seq=1,710,159,826,983 id=54dfb3d257b37029 ip=187.254.
30.196 udp=30303 tcp=30303
INFO [03-18|14:53:43.020] New local node record              seq=1,710,159,826,984 id=54dfb3d257b37029 ip=10.124.1
77.72 udp=30303 tcp=30303
INFO [03-18|14:53:43.499] NAT mapped port                    proto=TCP extport=30303 intport=30303 interface="UPNP
IGDv2-IP1"
INFO [03-18|14:53:44.007] NAT mapped port                    proto=UDP extport=30303 intport=30303 interface="UPNP
IGDv2-IP1"
```

El resultado de poner **Geth** es que estamos ejecutando el programa y nos iniciara un nodo local de Ethereum en la maquina o dicho de otra forma un servidor.

Para iniciar el nodo en el Net Sepolia utilizando las API necesarias ejecutamos el siguiente comando:

```
geth --syncmode "snap" --http --http.api  
"eth,net,web3,personal"  
--rpc.enabled deprecated personal --sepolia --allow-  
insecure-unlock
```

Este comando configura un nodo Geth para sincronizarse utilizando instantáneas, habilita un servidor HTTP con varias APIs, incluida la API personal, y permite el desbloqueo inseguro de cuentas para propósitos de desarrollo o pruebas.

```
Administrador: Windows Pow
PS C:\Users\ultr> geth --syncmode "snap" --http --http.api "eth,net,web3,personal" --rpc.enabled deprecatedpersonal --sepolia --allow-insecure-unlock
INFO [03-18|15:14:34.729] Starting Geth on Sepolia testnet
INFO [03-18|15:14:34.734] Maximum peer count                      ETH=50 total=50
INFO [03-18|15:14:34.739] Set global gas cap                      cap=50,000,000
INFO [03-18|15:14:34.739] Initializing the KZG library            backend=gokzg
INFO [03-18|15:14:34.761] Allocated trie memory caches            clean=154.00MiB dirty=256.00MiB
INFO [03-18|15:14:34.767] Using pebble as the backing database
INFO [03-18|15:14:34.767] Allocated cache and file handles        database=C:\Users\ultr\AppData\Local\Ethereum\sepolia\geth\chaindata cache=512.00MiB handles=8192
INFO [03-18|15:14:34.856] Opened ancient database                 database=C:\Users\ultr\AppData\Local\Ethereum\sepolia\geth\chaindata\ancient\chain reads only=false
INFO [03-18|15:14:34.858] State scheme set to already existing    scheme=hash
INFO [03-18|15:14:34.865] Initialising Ethereum protocol          network=11,155,111 dbversion=8
INFO [03-18|15:14:34.865] -----
INFO [03-18|15:14:34.865] Chain ID: 11155111 (sepolia)
INFO [03-18|15:14:34.865] Consensus: Beacon (proof-of-stake), merged from Ethash (proof-of-work)
INFO [03-18|15:14:34.866] Pre-Merge hard forks (block based):
INFO [03-18|15:14:34.866] - Homestead: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/homestead.md)
INFO [03-18|15:14:34.866] - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/tangerine-whistle.md)
INFO [03-18|15:14:34.866] - Spurious Dragon/1 (EIP 155): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/spurious-dragon-1.md)
INFO [03-18|15:14:34.866] - Spurious Dragon/2 (EIP 158): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/spurious-dragon-2.md)
INFO [03-18|15:14:34.866] - Byzantium: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/byzantium.md)
INFO [03-18|15:14:34.866] - Constantinople: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/constantinople.md)
INFO [03-18|15:14:34.866] - Petersburg: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/petersburg.md)
INFO [03-18|15:14:34.866] - Istanbul: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/istanbul.md)
INFO [03-18|15:14:34.866] - Muir Glacier: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/muir-glacier.md)
INFO [03-18|15:14:34.866] - Berlin: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/berlin.md)
```

Esto para poder iniciar un nodo Geth con ciertas configuraciones específicas:

geth: Este es el comando principal que inicia el nodo Ethereum utilizando el software Geth.

--syncmode "snap": Este parámetro especifica el modo de sincronización del nodo. En este caso, se está utilizando el modo "snap sync", que es un modo de sincronización rápida que permite sincronizar rápidamente el nodo con la red Ethereum.

--http: Habilita el servidor HTTP-RPC, lo que permite la comunicación con el nodo a través de solicitudes HTTP.

--http.api "eth,net,web3,personal": Este parámetro especifica las API que estarán disponibles a través del servidor HTTP-RPC. En este caso, se están habilitando las API de Ethereum, net, web3 y personal. Esto significa que podremos interactuar con estas API a través de solicitudes HTTP.

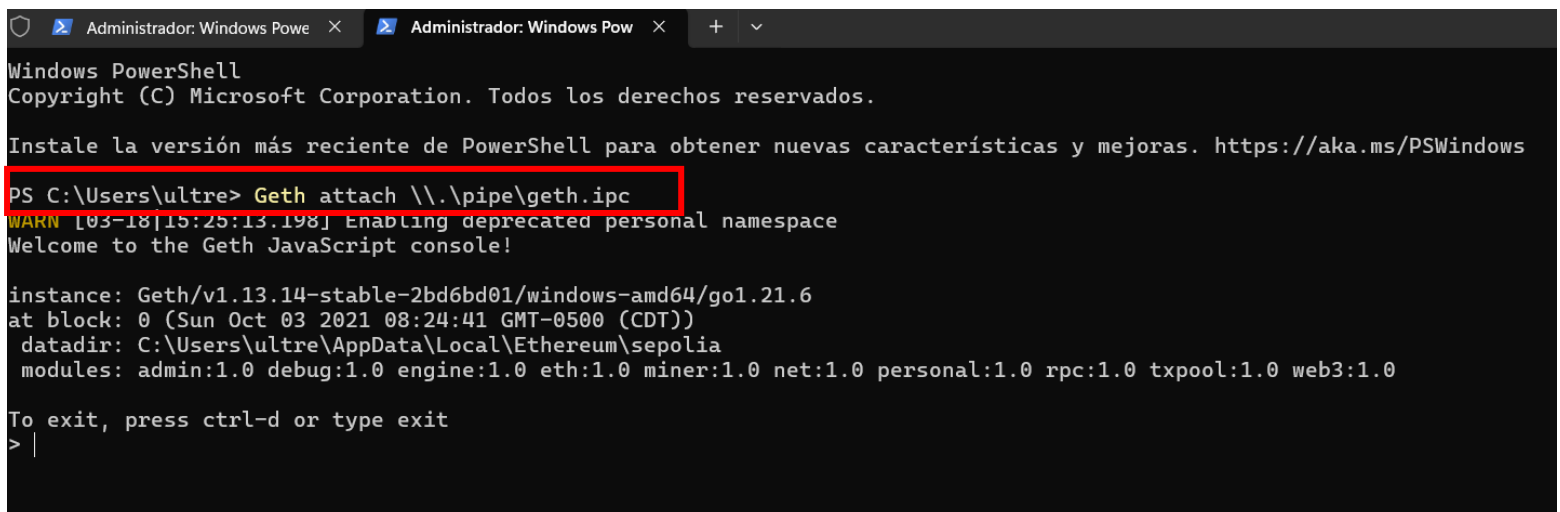
--rpc.enableddeprecatedpersonal: Este parámetro habilita la API personal de forma depreciada. La API personal proporciona funcionalidades relacionadas con la gestión de cuentas y claves privadas.

--sepolia: Este parámetro especifica que el nodo se conectará a la red de pruebas "sepolia". Esta es una red de prueba de Ethereum que puede ser utilizada para propósitos de desarrollo y pruebas **sin incurrir en costos de transacción**.

--allow-insecure-unlock: Este parámetro permite el desbloqueo de cuentas de forma insegura. En un entorno de producción, esto puede ser peligroso, ya que podría exponer tus cuentas a posibles ataques. Sin embargo, en un entorno de desarrollo o pruebas, puede ser útil para simplificar el proceso de desarrollo y pruebas.

3. Interactuar con geth utilizando la consola del nodo

Para abrir la consola del nodo abrimos una nueva terminal y ponemos el siguiente comando **Geth attach y la url que dio la IPC endpoint opened.**



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\ultre> Geth attach \\.\\pipe\\geth.ipc
[WARN] [03-18|15:25:13.198] Enabling deprecated personal namespace
Welcome to the Geth JavaScript console!

instance: Geth/v1.13.14-stable-2bd6bd01/windows-amd64/go1.21.6
at block: 0 (Sun Oct 03 2021 08:24:41 GMT-0500 (CDT))
datadir: C:\Users\ultre\AppData\Local\Ethereum\sepolia
modules: admin:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.0 personal:1.0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> |
```

En esta parte nos proporciona una interfaz interactiva que nos permitirá ejecutar comandos directamente en el nodo Ethereum en ejecución.

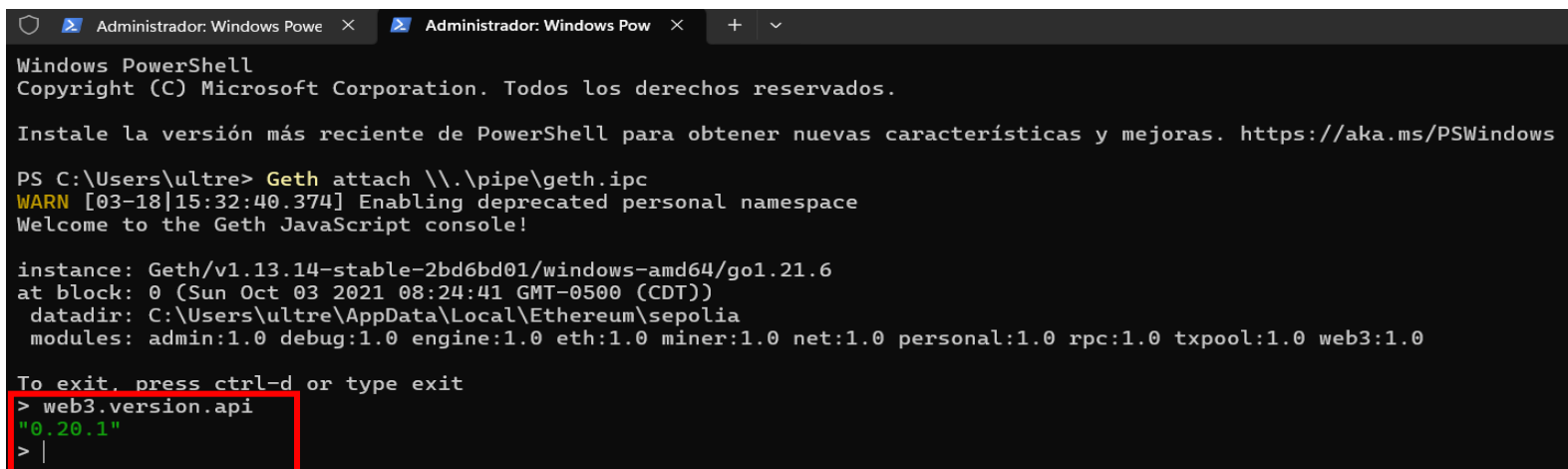
4. Buscar la siguiente información:

a. Versión del api

Comando: **web3.version.api**

- **web3.js** es una biblioteca de JavaScript que proporciona una interfaz para interactuar con nodos Ethereum.
- **web3.version.api** es una función de esta biblioteca que permite acceder a la versión de la API utilizada por web3.js.

Al ejecutar este comando, podríamos obtener información sobre la versión específica de la API que estamos utilizando.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\ultre> Geth attach \\.pipe\geth.ipc
WARN [03-18|15:32:40.374] Enabling deprecated personal namespace
Welcome to the Geth JavaScript console!

instance: Geth/v1.13.14-stable-2bd6bd01/windows-amd64/go1.21.6
at block: 0 (Sun Oct 03 2021 08:24:41 GMT-0500 (CDT))
datadir: C:\Users\ultre\AppData\Local\Ethereum\sepolia
modules: admin:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.0 personal:1.0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> web3.version.api
"0.20.1"
> |
```

b. Version de node

Comando: **web3.version.node**

- **web3.js** es una biblioteca de JavaScript que proporciona una interfaz para interactuar con nodos Ethereum.
- **web3.version.node** es una función de esta biblioteca que permite acceder a la versión del nodo Ethereum al que está conectada la instancia de web3.js.

Al obtener la versión del nodo Ethereum, podemos verificar qué implementación del cliente se está utilizando y también asegurarnos de que nuestra aplicación esté interactuando correctamente con el nodo

Ethereum deseado.

```
To exit, press ctrl-d or type exit
> web3.version.api
"0.20.1"
> web3.version.node
"Geth/v1.13.14-stable-2bd6bd01/windows-amd64/go1.21.6"
> |
```

c. Información del nodo

Comando: **admin.nodeInfo**

- **admin** es un módulo en Geth (cliente Ethereum) que proporciona funcionalidades relacionadas con la administración del nodo.
- **admin.nodeInfo** es un comando dentro de este módulo que permite acceder a información específica sobre el nodo Ethereum en el que se está ejecutando Geth.

Se utiliza en un entorno que tiene acceso a un nodo Ethereum con el módulo de administración habilitado. Al ejecutar este comando, nos devuelve información detallada sobre el nodo Ethereum, incluyendo detalles como la versión del software utilizado, el ID del nodo, la dirección IP y el puerto en el que está escuchando, entre otros.

```
To exit, press ctrl-d or type exit
> admin.nodeInfo
{
  enode: "enode://c298b03b3450bb0298ecd3b8912eb47dc5e6fda772bbcd078c1be1fe9175f61df09f61e59cb079eeadf9f088c3b40ae0f34eabf64212e34866a00bd5d6f31d9010.124.177.72:30303",
  enr: "enr:-Ka4QIUxHcsxKAW8yXue7M2Wx7nwwlcw_cykMKUlg01R58PmASooeXT6sF5oN9VvX15o8leg63AJUQLF5Cw9sAgpyGAY434FqCg2V0aMr3hP4zZueDGnrLgmlkgnY0gmlwhAp8sU1Jc2VjcDI1NmsxoQPCmLA7NF57Apjs07iRLrR9xeb9p3K7zQeMG-H-kXX2HYRzbnFwwIN0Y3CCdl-DdWRwgnZf",
  id: "baf6fc45691f40a1ca68af07cf03c86ae48188f07152fa7222afbfa8c5bbfdd",
  ip: "19.124.177.72",
  listenAddr: "[::]:30303",
  name: "geth/v1.13.14-stable-2bd6bd01/windows-amd64/go1.21.6",
  ports: {
    discovery: 30303,
    listener: 30303
  },
  protocols: {
    eth: {
      config: {
        berlinBlock: 0,
        byzantiumBlock: 0,
        cancunTime: 170665072,
        chainId: 11155111,
        constantinopleBlock: 0,
        daoForkSupport: true,
        eip150Block: 0,
        eip155Block: 0,
        eip158Block: 0,
        ethash: {},
        homesteadBlock: 0,
        istanbulBlock: 0,
        londonBlock: 0,
        mergeHetsplitBlock: 1735371,
        muirGlacierBlock: 0,
        petersburgBlock: 0,
        shanghaiTime: 1677557088,
        terminalTotalDifficulty: 17000000000000000,
        terminalTotalDifficultyPassed: true
      },
      difficulty: 131072,
      genesis: "0x25a5cc106eea7138acab33231d7160d69cb777ee0c2c553fcd5f5138993e6dd9",
      head: "0x25a5cc106eea7138acab33231d7160d69cb777ee0c2c553fcd5f5138993e6dd9",
      network: 11155111
    },
    snap: {}
  }
}
```

d. Cantidad de peernodes

Comando: **net.peerCount**

Este comando se utiliza en un entorno que tiene acceso a un nodo Ethereum con el módulo net habilitado. Al ejecutar este comando, nos devuelve el número de peers conectados al nodo Ethereum en ese preciso momento.

```
>  
> net.peerCount  
3  
> |
```

e. datos de red

Comando: **eth.syncing**

Este comando devuelve información sobre si el nodo está sincronizado con la red o si está en proceso de sincronización. Si el nodo está sincronizado, devolverá false. Sin embargo, si el nodo está en proceso de sincronización, devolverá un objeto que contiene información sobre el progreso de la sincronización, como el bloque actual, el bloque más alto y el bloque más alto conocido en la red.

```
> eth.syncing  
{  
  currentBlock: 0,  
  healedBytecodeBytes: 0,  
  healedBytecodes: 0,  
  healedTrieNodeBytes: 0,  
  healedTrieNodes: 0,  
  healingBytecode: 0,  
  healingTrieNodes: 0,  
  highestBlock: 0,  
  startingBlock: 0,  
  syncedAccountBytes: 0,  
  syncedAccounts: 0,  
  syncedBytecodeBytes: 0,  
  syncedBytecodes: 0,  
  syncedStorage: 0,  
  syncedStorageBytes: 0,  
  txIndexFinishedBlocks: 0,  
  txIndexRemainingBlocks: 1  
}  
> |
```

f. información de los peers al menos 2

Comando: **admin.peers**

Este comando me permite tener una visión clara de la conectividad de mi nodo Ethereum con otros nodos en la red. Puedo ver cuántos pares están activos y obtener detalles específicos sobre ellos, lo que es útil para monitorear la salud y la actividad de la red, así como para diagnosticar posibles problemas de conectividad.

Con esta información, puedo asegurarme de que mi nodo esté interactuando adecuadamente con la red Ethereum.

```
To exit, press ctrl-d or type exit
> admin.peers
[{"caps": ["eth/68", "snap/1"],
  enode: "enode://6375d0f3905b3935b944c4c8c0b815cb7a371fd318ccc95ec60fafddce7c230842a578a8946ae65f4fac71764cfff51307705466938b1790bf4a1213483e5d4f@176.235.209.218:30303",
  enr: "enr:-ROHQMJ_Agtmgk4EcsczcsGrantZHLwVP1I483h0TINy65XJ0VhsiMeg-FPLSLR2fvhAA_BEBhXfgcsFvUly_uTd8JsyGAY2wd18Pg2V0aHfGhIjPgdaAgmLkgnY8gnlwhLDr0dqJc2VjcdI1NmsxoQKjddDzKfs5NbLExHj8uBXLcjcf0xjMyV7Gd6_dznmjCIRzbnFwIN0Y3CCd1-OdwWgnZf",
  id: "05d3a7631744265b9f010a7f240823b22e037c637edeb206d052875038c9e338",
  name: "Geth/v1.13.12-stable/linux-amd64/go1.21.7",
  network: {
    inbound: false,
    localAddress: "192.168.0.17:56156",
    remoteAddress: "176.235.209.218:30303",
    static: false,
    trusted: false
  },
  protocols: {
    eth: {
      version: 68
    },
    snap: {
      version: 1
    }
  }
}, {
  caps: ["eth/68", "snap/1"],
  enode: "enode://143e11fb766781d22d92a2e33f8f104cddae4d11a22295ed1fdb6638de96a6ce65f5b7c964ba3763bba27961738fef7d3ecc739268f3e5e771fb4c87b6234ba@146.198.1.103:30303",
  id: "8c3148ab1c5ea102b9f04345898d8b0b08b1be17feb63196fd440ca14ccc1bd",
  name: "Geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.7",
  network: {
    inbound: false,
    localAddress: "192.168.0.17:55882",
    remoteAddress: "146.198.1.103:30303",
    static: false,
    trusted: false
  },
  protocols: {
    eth: {
      version: 68
    },
    snap: {
      version: 1
    }
  }
}, {
  caps: ["eth/68", "snap/1"],
  enode: "enode://10d52eff032285fcef19097f35ca8f77ba0eadfffd769a107e895d8b2b8f8ae6341630c645c38f5df6e67597c83090ced3d9c5764e8622a26587b083b028e88139.59.49.206:30303",
  enr: "enr:-ROHQMJ_Agtmgk4EcsczcsGrantZHLwVP1I483h0TINy65XJ0VhsiMeg-FPLSLR2fvhAA_BEBhXfgcsFvUly_uTd8JsyGAY2wd18Pg2V0aHfGhIjPgdaAgmLkgnY8gnlwhLDr0dqJc2VjcdI1NmsxoQKjddDzKfs5NbLExHj8uBXLcjcf0xjMyV7Gd6_dznmjCIRzbnFwIN0Y3CCd1-OdwWgnZf",
  id: "f9a3a33f4b5d1ac15cf1d6c11189c964530d165a92cbf6452655de7e8736afe",
  name: "Geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.7",
  network: {
    inbound: false,
    localAddress: "192.168.0.17:59497",
    remoteAddress: "139.59.49.206:30303",
    static: false,
    trusted: false
  },
  protocols: {
    eth: {
      version: 68
    },
    snap: {
      version: 1
    }
  }
}]
>
```

g. Conversiones entre ether

Para convertir una cantidad de Ether a Wei, usamos el siguiente comando **web3.toWei(1, 'ether')**, especificando el valor en Ether y la unidad "wei":

```
> web3.toWei(1, 'ether')  
"1000000000000000000000"  
>  
>
```

Esto convertirá 1 Ether a su equivalente en Wei.

Wei es la unidad más pequeña de ether en Ethereum, donde 1 ether equivale a 10^{18} wei. Esta conversión es útil cuando se trabaja con cantidades de ether en operaciones inteligentes o contratos inteligentes, ya que Ethereum generalmente maneja valores en wei.

Entonces, al ejecutar este comando, obtengo el valor numérico equivalente de 1 ether en wei, que es 1000000000000000000 (es decir, 1 seguido de 18 ceros).

Para convertir una cantidad de Wei a Ether, usamos el siguiente comando **web3.fromWei(1000000000000000000, 'ether')**, especificando el valor en Wei y la unidad "ether":

```
> web3.fromWei(100000000000000000000, 'ether')
"1"
> |
```

Esto convertirá 1000000000000000000 Wei (que es 1 Ether en Wei) a su equivalente en Ether.

Cuando ejecuto `web3.fromWei(1000000000000000000, 'ether')` desde mi entorno de desarrollo o script que utiliza la biblioteca web3.js, estoy convirtiendo una cantidad de wei, en este caso 1000000000000000000 wei, a su equivalente en ether.

Wei es la unidad más pequeña de ether en Ethereum, mientras que ether es la unidad más grande. En Ethereum, 1 ether es igual a 10^{18} wei.

Entonces, al ejecutar este comando, obtengo el valor numérico equivalente de 10000000000000000000 wei en ether, que es 1.

h. Al menos otros 5 comandos de preferencia

Crear un nuevo par de claves (dirección y clave privada):

Comando: `personal.newAccount()`

```
To exit, press ctrl-d or type exit
> personal.newAccount()
Passphrase:
Repeat passphrase:
"0xEbfbE340b101475419F2dF19Ae09DD9a15F41234"
> |
```

Al llamar a esta función, el nodo Ethereum generará una nueva clave privada y la asociará con una nueva dirección pública. Esta dirección pública se utilizará como identificador único de la cuenta en la red Ethereum.

Listar todas las cuentas disponibles en el nodo:

Comando: `eth.accounts`

```
> eth.accounts
["0xebfbe340b101475419f2df19ae09dd9a15f41234"]
> |
```

Cuando ejecuto **eth.accounts** desde mi nodo Ethereum, obtengo una lista de todas las cuentas que están actualmente disponibles en el nodo. Estas cuentas pueden haber sido creadas previamente utilizando el comando **personal.newAccount()** u obtenidas de otras fuentes, como importar claves privadas.

Cada cuenta en la lista está identificada por su dirección pública, que es única en la red Ethereum. Estas direcciones se utilizan para enviar y recibir ether, así como para interactuar con contratos inteligentes y realizar otras operaciones en la red.

Obtener el balance de una cuenta específica:

Comando: **web3.eth.getBalance('0xDireccionDeCuenta')**

```
> web3.eth.getBalance('0xebfbe340b101475419f2df19ae09dd9a15f41234')  
0  
> |
```

Al ejecutarlo, estoy consultando el saldo de ether de una cuenta específica en la red Ethereum.

En este comando, **'0xDireccionDeCuenta'** debe ser reemplazado por la dirección pública de la cuenta de la cual quiero obtener el saldo. Esta dirección identifica de manera única la cuenta en la red Ethereum.

Al ejecutar este comando, se envía una solicitud al nodo Ethereum para obtener el saldo actual de la cuenta especificada.

Verificar el estado de sincronización del nodo:

Comando: **eth.syncing**

```
> eth.syncing  
{  
  currentBlock: 0,  
  healedBytecodeBytes: 0,  
  healedBytecodes: 0,  
  healedTrienodeBytes: 0,  
  healedTrienodes: 0,  
  healingBytecode: 0,  
  healingTrienodes: 0,  
  highestBlock: 0,  
  startingBlock: 0,  
  syncedAccountBytes: 0,  
  syncedAccounts: 0,  
  syncedBytecodeBytes: 0,  
  syncedBytecodes: 0,  
  syncedStorage: 0,  
  syncedStorageBytes: 0,  
  txIndexFinishedBlocks: 0,  
  txIndexRemainingBlocks: 1  
}  
> |
```

Este comando devuelve información sobre si el nodo está sincronizado

con la red o si está en proceso de sincronización. Si el nodo está sincronizado, devolverá false. Sin embargo, si el nodo está en proceso de sincronización, devolverá un objeto que contiene información sobre el progreso de la sincronización, como el bloque actual, el bloque más alto y el bloque más alto conocido en la red.

Desbloquear una cuenta para permitir transacciones sin contraseña durante un período de tiempo específico:

Comando: `personal.unlockAccount('0xDireccionDeCuenta', 'Contraseña', 300)`

```
> personal.unlockAccount('0xebfbe340b101475419f2df19ae09dd9a15f41234', 'Barrabasito20', 300)
true
> |
```

Cuando ejecuto `personal.unlockAccount('0xDireccionDeCuenta', 'Contraseña', 300)` desde mi nodo Ethereum, estoy desbloqueando una cuenta específica en el nodo para permitir transacciones sin tener que ingresar la contraseña manualmente cada vez que se realiza una transacción.

- **'0xDireccionDeCuenta'** es la dirección pública de la cuenta que quiero desbloquear.
- **'Contraseña'** es la contraseña asociada con la cuenta que quiero desbloquear.
- **300** es la duración en segundos durante la cual la cuenta permanecerá desbloqueada. En este caso, la cuenta se mantendrá desbloqueada durante 300 segundos (5 minutos).

Aquí más comandos:

eth.getBlock(blockHashOrBlockNumber): Devuelve información sobre un bloque específico en la cadena Ethereum, dado su hash o número de bloque.

eth.getTransaction(transactionHash): Devuelve información sobre una transacción específica en la cadena Ethereum, dada su hash.

eth.getCode(address): Devuelve el código del contrato inteligente almacenado en una dirección específica en la cadena Ethereum.

eth.sendTransaction(transactionObject): Envía una transacción a la red Ethereum para ser procesada. El objeto de transacción contiene detalles como el destinatario, el valor a enviar y los datos adicionales.

eth.mining: Devuelve un valor booleano que indica si el nodo está actualmente minando nuevos bloques en la cadena Ethereum.

5. Conectarse Al JSON RPC API y realizar lo siguiente:

1. Levantar el API con los modulos de net, eth, web3, personal,admin.

```
PS C:\Users\ultre> geth --syncmode "snap" --http --http.api "eth,net,web3,personal" --rpc.enabled deprecatedpersonal --sepolia --allow-insecure-unlock
```

```
INFO [03-18|20:11:02.066] Chain post-merge, sync via beacon client
INFO [03-18|20:11:02.066] Gasprice oracle is ignoring threshold set threshold=2
WARN [03-18|20:11:02.070] Unclean shutdown detected
WARN [03-18|20:11:02.071] Engine API enabled
INFO [03-18|20:11:02.071] Starting peer-to-peer node
INFO [03-18|20:11:02.112] New local node record
WARN [03-18|20:11:02.112] Deprecated personal namespace activated
INFO [03-18|20:11:02.112] Started P2P networking
79eeade9f088c3b40ae0f34eabf64212e34868a0bd5d6f31d9@127.0.0.1:30303
INFO [03-18|20:11:02.113] IPC endpoint opened
INFO [03-18|20:11:02.113] Loaded Geth secret file
INFO [03-18|20:11:02.115] HTTP server started
INFO [03-18|20:11:02.120] WebSocket enabled
INFO [03-18|20:11:02.121] HTTP server started
INFO [03-18|20:11:04.874] New local node record
INFO [03-18|20:11:05.063] NAT mapped port
INFO [03-18|20:11:12.316] Looking for peers
INFO [03-18|20:11:22.586] Looking for peers
INFO [03-18|20:11:32.724] Looking for peers
WARN [03-18|20:11:37.080] Post-merge network, but no beacon client
INFO [03-18|20:11:43.069] Looking for peers
```

```
ETH=50 total=50
cap=50,000,000
backend=gokzg
clean=154.00MiB dirty=256.00MiB
database=C:\Users\ultre\AppData\Local\Ethereum\sepolia\geth\chaindata cache=512.00MiB han
dles=8192
database=C:\Users\ultre\AppData\Local\Ethereum\sepolia\geth\chaindata\ancient\chain reado
nly=false
scheme=hash
network=11,155,111 dbversion=8
```

```
booted=2024-03-14T06:25:09-0500 age=4d13h45m
protocol=eth
instance=Geth/v1.13.14-stable-2bd6bd01/windows-amd64/go1.21.6
seq=1,710,334,433,923 id=baf6fc45691f40a1 ip=127.0.0.1 udp=30303 tcp=30303
self=enode://c298b03b3454bb0298ecd3b8912eb47dc5e6fda772bbcd078c1be1fe9175f61df09f61e59cb0
url=\\.\pipe\geth.ipc
path=C:\Users\ultre\AppData\Local\Ethereum\sepolia\geth\jwtsecret secret=0x180f9bde
endpoint=127.0.0.1:8545 auth=false prefix= cors= vhosts=localhost
url=ws://127.0.0.1:8551
endpoint=127.0.0.1:8551 auth=true prefix= cors=localhost vhosts=localhost
seq=1,710,334,433,924 id=baf6fc45691f40a1 ip=10.124.177.72 udp=30303 tcp=30303
proto=TCP extport=30303 intport=30303 interface="UPNP IGDv2-IP1"
proto=UDP extport=30303 intport=30303 interface="UPNP IGDv2-IP1"
peercount=0 tried=31 static=0
peercount=0 tried=29 static=0
peercount=0 tried=37 static=0
peercount=0 tried=35 static=0
```

Ejecutamos lo visto anteriormente para poder levantar el servidor con las APIS necesarias para poder realizar peticiones mediante Postman.

2. Ejecutar al menos 2 metodos de cada modulo con peticiones POST

Web3

The screenshot displays a REST client interface with a sidebar on the left listing various endpoints under 'Ethereum JSON-RPC'. The main panel shows a POST request to '127.0.0.1:8545' with the following JSON body:

```
1 {
2   "jsonrpc": "2.0",
3   "method": "web3_clientVersion",
4   "params": [],
5   "id": 1
6 }
```

The response body is shown below, indicating the client version:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 1,
4   "result": "Geth/v1.13.14-stable-2bd6bd01/windows-amd64/go1.21.6"
5 }
```

Ponemos la dirección de IP y puerto donde podremos acceder o tener esa comunicación con el servidor o nodo.

El método **web3_clientVersion** es una función que se utiliza en JSON-RPC para obtener la versión del cliente (software del nodo) que estás utilizando para interactuar con la red Ethereum.

Cuando realizamos una solicitud JSON-RPC que incluye web3_clientVersion, el nodo Ethereum responderá con una cadena que representa la versión del software del cliente que está ejecutando.

The screenshot shows a REST client interface with a sidebar on the left listing various endpoints under 'Ethereum JSON-RPC'. The main panel displays a POST request to '127.0.0.1:8545' with the path 'web3 / sha3'. The request body is a JSON object:

```
{  "jsonrpc": "2.0",  "method": "web3_sha3",  "params": ["0x68656c6c6f207766726c64"],  "id": 64}
```

. The response body is a JSON object:

```
{  "jsonrpc": "2.0",  "id": 64,  "result": "0x47173285a8d7341e5e972fc677286384f802f8ef42a5ec5f03bbfa254cb01fad"}
```

.

En JSON-RPC, el método **web3_sha3** se utiliza para realizar cálculos criptográficos en un nodo Ethereum. Este método toma un único parámetro, que es una cadena hexadecimal que se desea hashear, y devuelve el hash SHA3 correspondiente como una cadena hexadecimal de 32 bytes.

Net

The screenshot shows a REST client interface with a sidebar on the left listing various endpoints under 'Ethereum JSON-RPC'. The main panel displays a POST request to '127.0.0.1:8545' with the path 'net / version'. The request body is a JSON object:

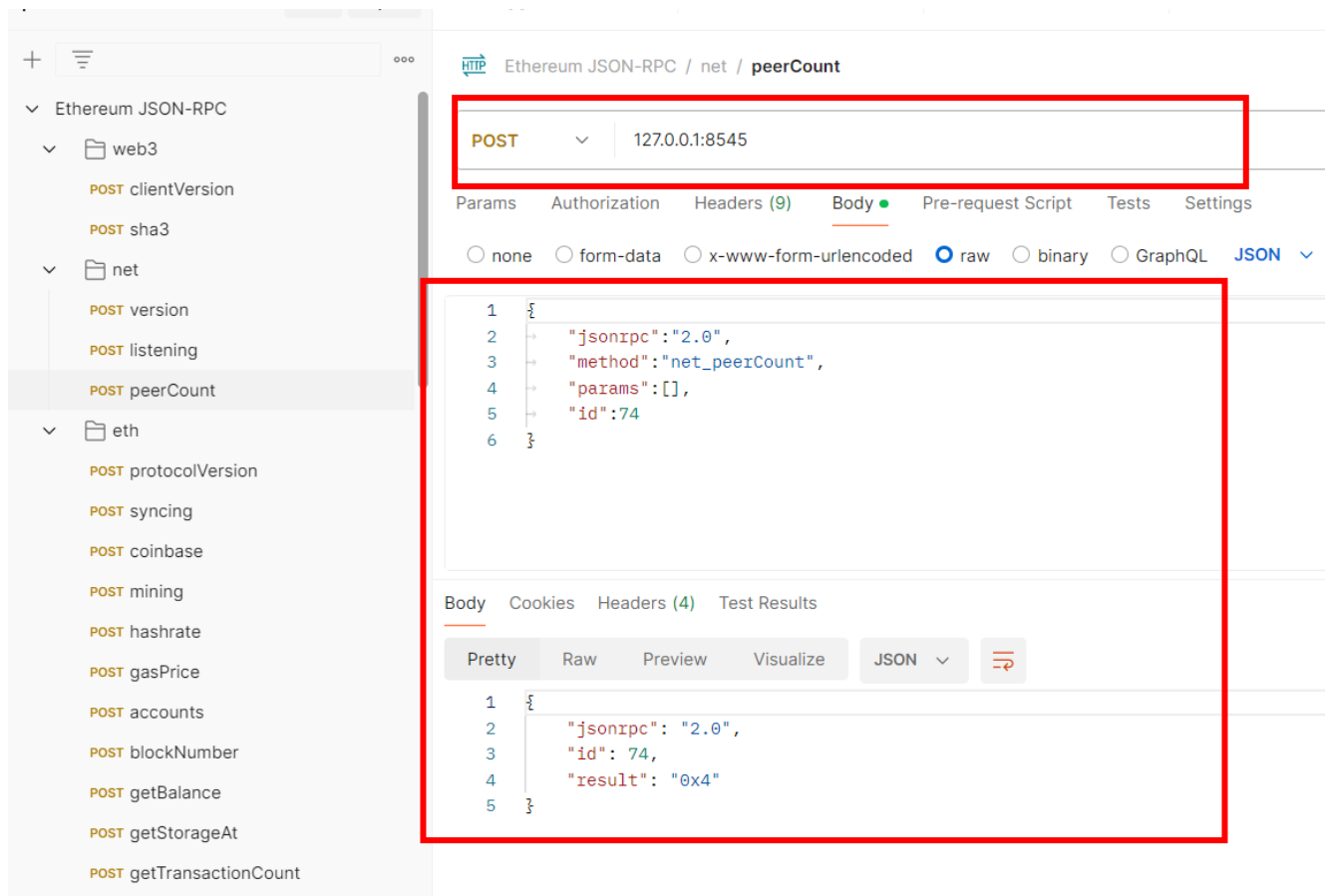
```
{  "jsonrpc": "2.0",  "method": "net_version",  "params": [],  "id": 67}
```

. The response body is a JSON object:

```
{  "jsonrpc": "2.0",  "id": 67,  "result": "11155111"}
```

.

El método **net_version** es una función de la interfaz JSON-RPC utilizada para obtener el número de versión de la red Ethereum a la que está conectado un nodo Ethereum.



El método **net_peerCount** es otro método utilizado en JSON-RPC en Ethereum para obtener el número de pares (peers) conectados al nodo Ethereum al que se está enviando la solicitud.

Cuando llamamos a `net_peerCount` a través de JSON-RPC, el nodo Ethereum responderá con un número entero que representa la cantidad de pares (nodos) con los que está conectado el nodo actual.

Eth

The screenshot displays a REST client interface with a sidebar on the left listing various Ethereum JSON-RPC endpoints. The main panel shows a POST request to `127.0.0.1:8545` with the following JSON body:

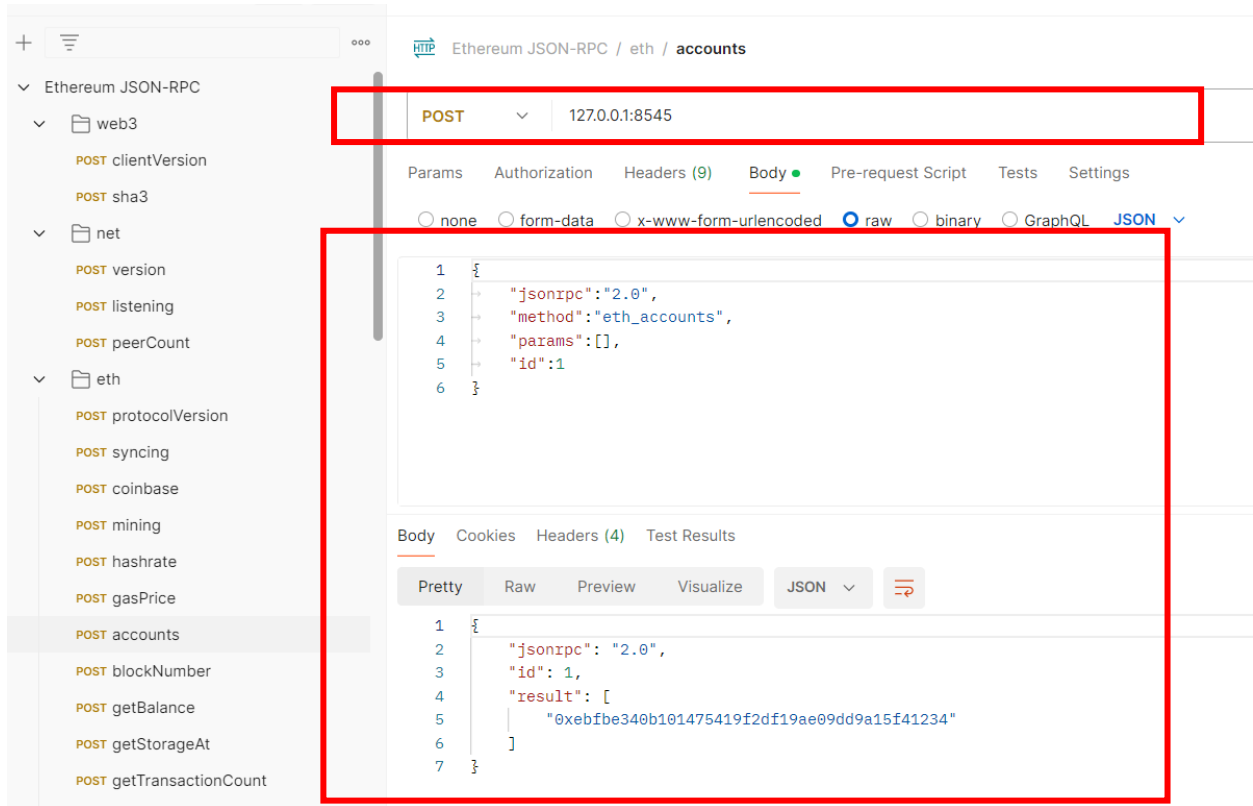
```
1 {
2   "jsonrpc": "2.0",
3   "method": "eth_protocolVersion",
4   "params": [],
5   "id": 67
6 }
```

The response is shown in the 'Body' tab, indicating an error:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 67,
4   "error": {
5     "code": -32601,
6     "message": "the method eth_protocolVersion does not exist/is not available"
7   }
8 }
```

El método **eth_protocolVersion** es otro de los métodos disponibles en JSON-RPC para interactuar con nodos Ethereum. Este método se utiliza para obtener la versión del protocolo que el nodo Ethereum está utilizando para comunicarse con otros nodos en la red Ethereum.

Cuando se envía una solicitud JSON-RPC que incluye `eth_protocolVersion`, el nodo Ethereum responderá con un número entero que representa la versión del protocolo que está utilizando actualmente.



El método **eth_accounts** es una función utilizada en JSON-RPC para obtener una lista de todas las cuentas administradas por el nodo Ethereum al que estás conectado.

Cuando envías una solicitud JSON-RPC que incluye `eth_accounts`, el nodo Ethereum responderá con un array que contiene las direcciones de todas las cuentas disponibles en ese nodo.

6. Crear una cuenta

Comando: **personal.newAccount()**

Este comando solicitará al nodo que genere una nueva clave privada y la asocie con una nueva dirección pública. La dirección pública resultante es única en la red Ethereum y servirá como identificador para esta nueva cuenta.

Después de ejecutar este comando, se me pedirá que ingrese una contraseña para proteger la nueva cuenta. Esta contraseña será necesaria para realizar transacciones o cualquier otra operación que requiera el uso de esta cuenta.

Una vez creada la cuenta, puedo usarla para recibir ether, enviar ether a otras cuentas, interactuar con contratos inteligentes y realizar otras operaciones en la red Ethereum.

```
To exit, press ctrl-d or type exit
> personal.newAccount()
Passphrase:
Repeat passphrase:
"0xEbfbE340b101475419F2dF19Ae09DD9a15F41234"
> |
```

Aquí se muestra cómo cree la cuenta y me pidió una contraseña para protegerla, ingresándola dos veces para su confirmación después me arrojo la llave pública esta es la que utilizare para recibir fondos y enviar.

Comando: **eth.accounts**

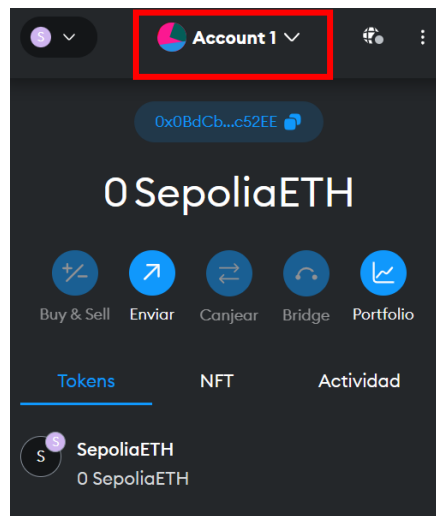
Cuando ejecuto este comando, el nodo devuelve una lista de direcciones públicas de todas las cuentas que están almacenadas localmente en el nodo.

Estas cuentas pueden haber sido creadas utilizando el comando **personal.newAccount()** o importadas desde otras fuentes.

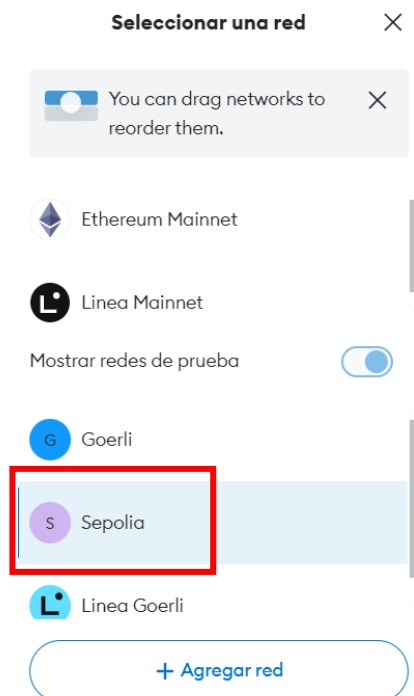
```
> eth.accounts
["0xebfbe340b101475419f2df19ae09dd9a15f41234"]
> |
```

7. Importar su cuenta a metamask

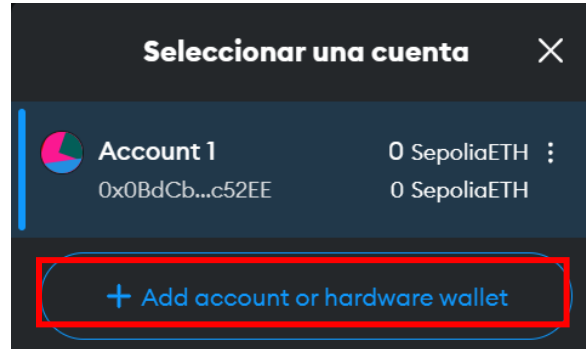
Primeramente, agregamos la extensión a nuestro navegador, abrimos la extensión, creamos una cuenta en MetaMask para iniciar sesión y nos generara un Account 1.



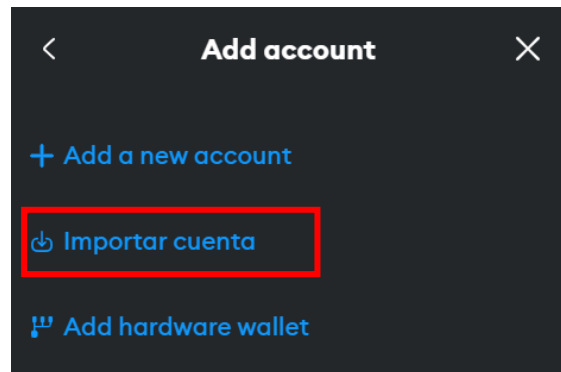
Seleccionamos la red de Sepolia en redes de prueba.



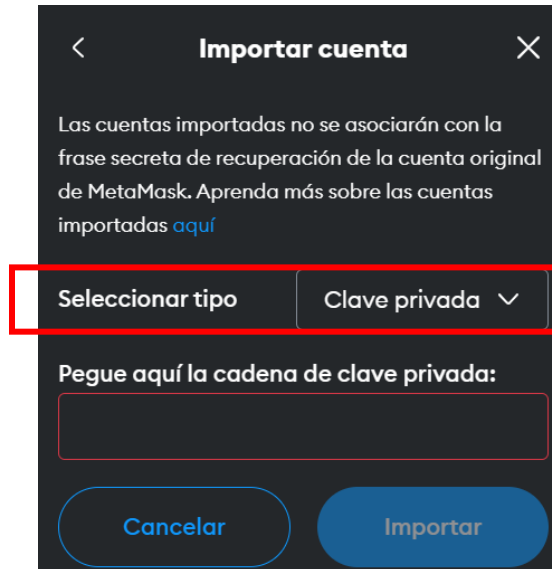
En la lista desplegable de cuentas para seleccionar o agregar cuentas, abrimos y seleccionamos "add account or hardware wallet".



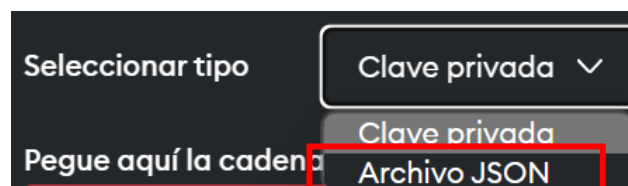
Al seleccionar, nos saldrán las opciones: Agregar una nueva cuenta, Importarla y Add hardware wallet.



En este caso vamos a importar una cuenta, por lo tanto, seleccionamos "Importar Cuenta".



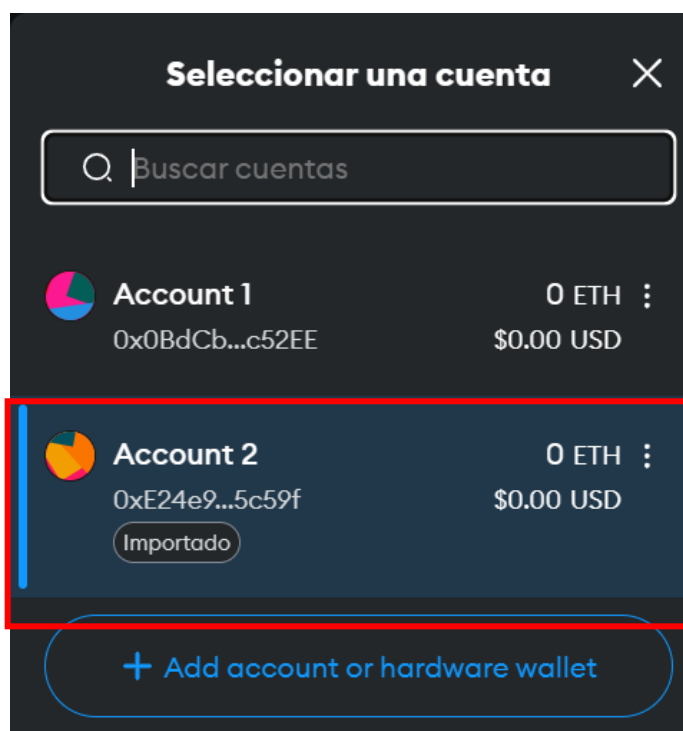
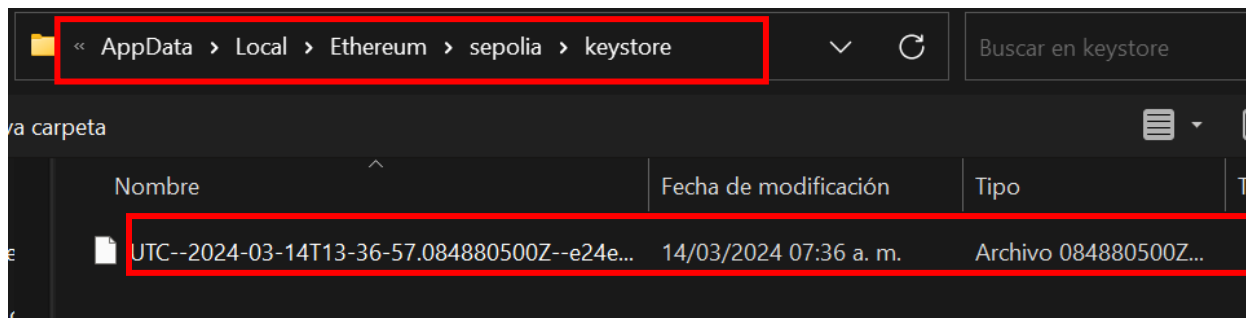
Una vez estando ahí, nos dará la opción de seleccionar el tipo de importación si por Clave Privada o Archivo JSON.



En este caso seleccionamos el tipo "Archivo JSON".



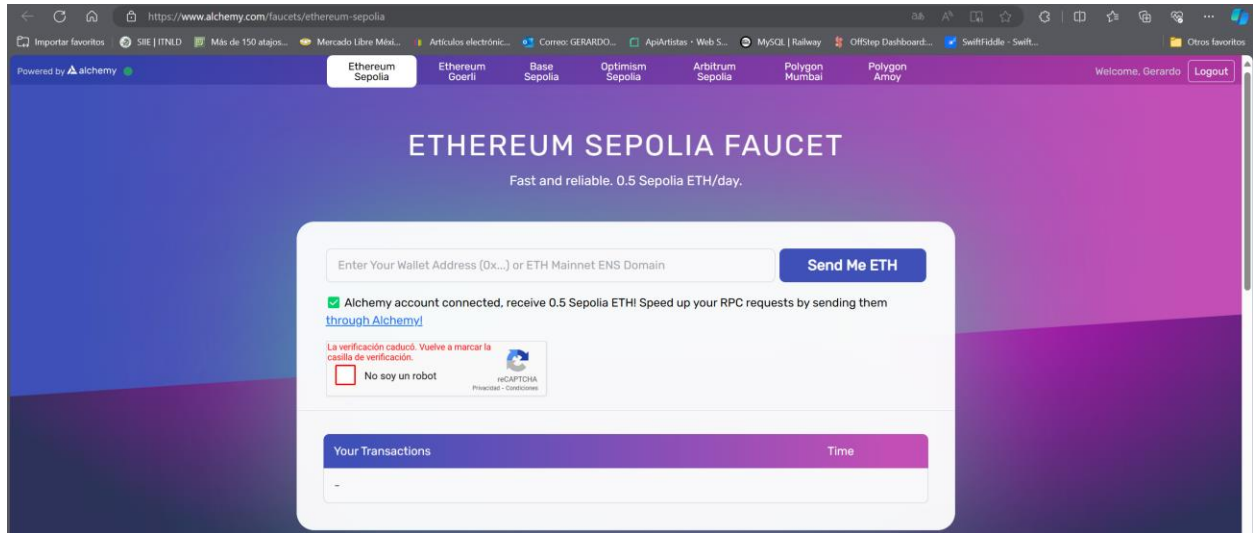
Con esta opción podemos abrir el archivo JSON y lo importamos de nuestro directorio.



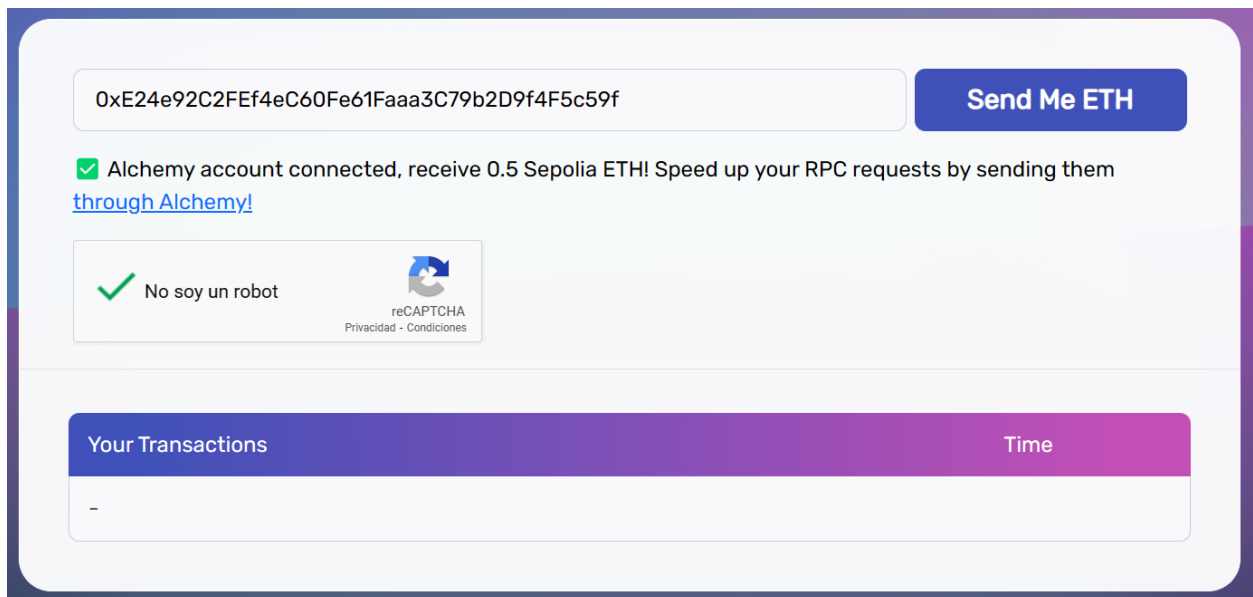
De esta forma ya se agregaría la cuenta que importamos.

8. Obtener SepoliaETH utilizando un Faucet

En la página [Ethereum Sepolia Faucet \(alchemy.com\)](https://www.alchemy.com/faucets/ethereum-sepolia) creamos una cuenta, tenemos que ingresar la clave publica de mi cuenta previamente creada e importada a MetaMask para que me envíe SepoliaETH. Así esta página nos dará gratuitamente 0.5 Sepolia ETH cada 24 horas.



Al ingresar la llave publica, verificamos el recaptcha y seleccionamos SEND ME ETH.



Al abrir la extensión, se verá reflejado en la cuenta importada los 0.5 SepoliaETH obtenidos al utilizar ETHEREUM SEPOLIA FAUCET.

